SliceMatic

Stage 1 Submission

Product Requirements Document & Business Unit Economics

FDE Programme · Batch 2487 · PizzaFlow Applied Project

June 25, 2026

Om · Febin · Alok · Guru · Rahul

github.com/omiiii21/Slicematic_Agentic_Flow

SliceMatic: Product Requirements Document (PRD)

Team: Om, Febin, Alok, Guru, Rahul · **Repo:** `Slicematic_Agentic_Flow` · **Date:** June 2026

---

1. Product Vision

SliceMatic's ordering system is a digital, single-outlet pizza-ordering platform for a delivery-first QSR in New Ashok Nagar, Delhi. It replaces error-prone, slow telephone order-taking with a guided, validated, self-service flow that walks a customer from intake to a GST-correct bill and a confirmed payment in under two minutes — and writes every order to a durable log so the owner can run the business on data instead of memory. It serves urban 18–35 customers within a 4 km radius and the outlet staff who currently transcribe orders by hand. The problem it replaces: missed/garbled phone orders, manual price and discount maths, no record of what sold, and zero analytics.

---

2. Functional Requirements

Each requirement is written precisely enough to build from. IDs are referenced by the technical spec (`SPEC.md`) and the test suite.

FR-1 Customer onboarding

- FR-1.1 Collect **name**: letters and spaces only, 2–40 characters.
- FR-1.2 Collect **phone**: exactly 10 digits, first digit ∈ {6,7,8,9} (Indian mobile).
- FR-1.3 On invalid input, show a **specific** error and re-prompt on the same step; never advance, never crash.
- FR-1.4 Record a session timestamp when intake begins.

FR-2 Quantity selection

- FR-2.1 Accept integers **1–10** only. Reject 0, negatives, floats, and words.
- FR-2.2 Auto-apply a **10% discount when qty ≥ 5**; the discount line must appear on the bill.
- FR-2.3 Reject qty > 10 with an explanation (outlet capacity is 10 pizzas/order).

FR-3 Menu browsing (loaded from files / DB)

- FR-3.1 Load `Types_of_Base.txt` , `Types_of_Pizza.txt` , `Types_of_Toppings.txt` at startup. **No hardcoded menu.**
- FR-3.2 Display each menu as a numbered list with name + INR price.
- FR-3.3 Accept selection **by item number only**; reject out-of-range numbers, letters, and empty input.
- FR-3.4 Customer composes one pizza = one **base** + one **pizza** + one **topping**.

FR-4 Pricing engine

- FR-4.1 Per-unit price = base + pizza + topping.
- FR-4.2 Subtotal = per-unit × qty.
- FR-4.3 Discount = 10% of subtotal when qty ≥ 5, else 0.

- FR-4.4 GST = **18% of the post-discount subtotal** (discount first, then GST).
- FR-4.5 Final payable = post-discount subtotal + GST. All money rounded to 2 decimals.

FR-5 Order summary & bill

- FR-5.1 Itemised bill: each component, unit price, qty, line amount.
- FR-5.2 Show subtotal, discount (if any), post-discount, GST, and total payable.
- FR-5.3 Rendered in a structured component (table / HTML), **not** a plain text box, with currency symbol and aligned columns.

FR-6 Payment flow

- FR-6.1 Exactly three modes: **1 Cash · 2 Card · 3 UPI**.
- FR-6.2 Reject any other selection with a retry prompt.
- FR-6.3 Show a mode-specific confirmation message.

FR-7 Order persistence

- FR-7.1 Append every completed order to `orders_log.txt` (Stage 2) / Supabase tables (Stage 3).
- FR-7.2 Each record includes: timestamp, name, phone, item selections, unit prices, qty, subtotal, discount, GST, total, payment mode.
- FR-7.3 Format: one order block, pipe-separated `key=value` fields, blank line between orders (parseable + human-readable).

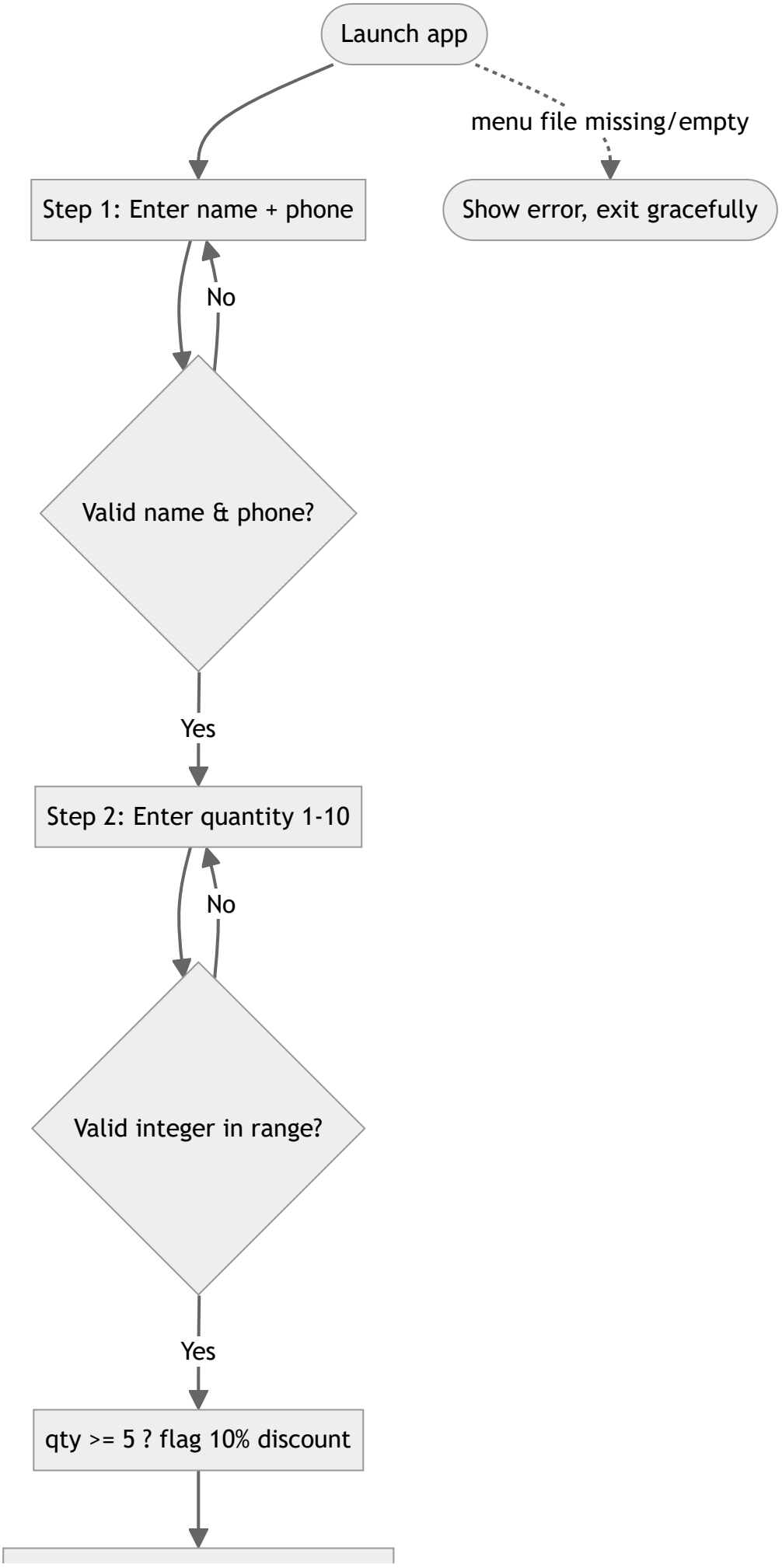
---

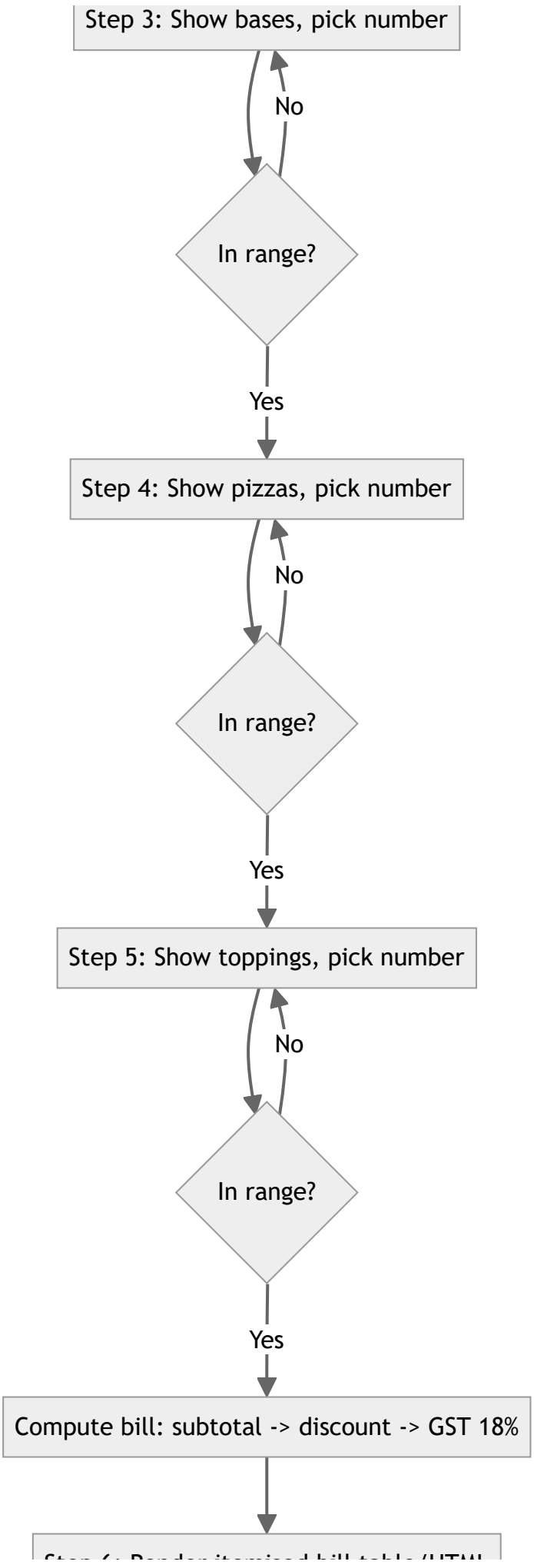
3. Non-Functional Requirements

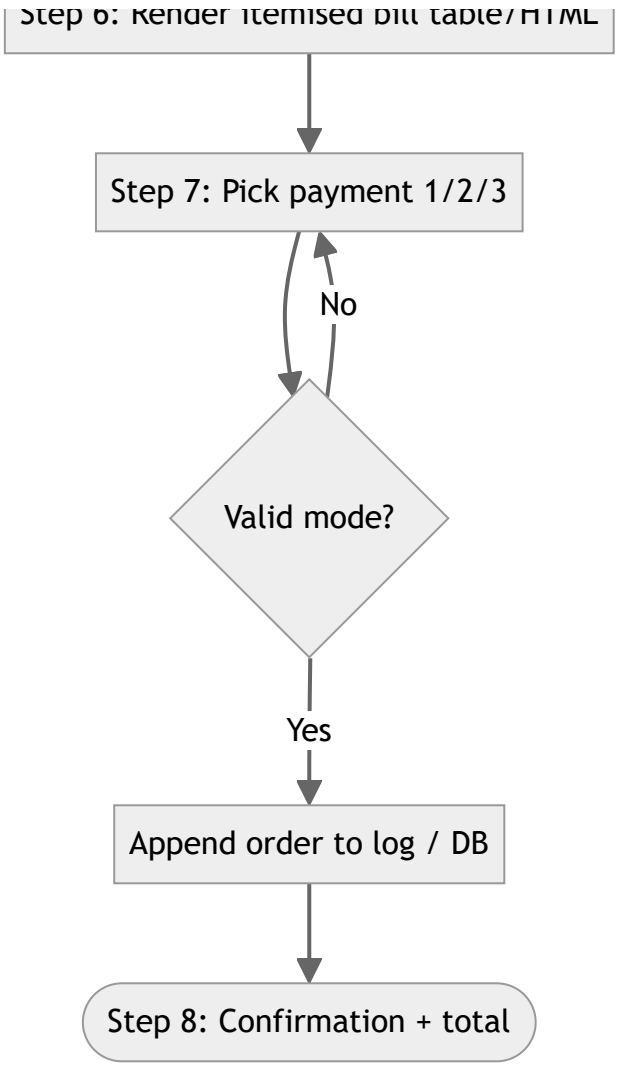
Area	Rule
Validation	Every input validated before use; see the validation table in `SPEC.md` (one regex/rule per field).
Error messages	Specific and actionable ("Indian mobile numbers must start with 6, 7, 8 or 9"), never a stack trace.
Edge-case safety	The 8 grader edge cases must be handled with **zero unhandled exceptions**. Verified by `tests/test_core.py` .
Menu-swap safety	Parser must survive the grader swapping the files: strip whitespace/BOM, skip rows with missing fields or non-numeric price, error cleanly if a file is missing or has no valid rows.
Orders log format	UTF-8, append-only, one block/order, blank-line separated, `key=value` pipe-joined fields.
Graceful failure	Missing/empty menu file → visible message and stop, not a crash.
Determinism	Same inputs → same bill, to the paisa, matching the reference sample (Rs.3,594.87).
Statelessness of logic	Pricing/validation live in a pure module (`core.py`) reused unchanged by the Stage 3 backend.

---

4. User Flow Diagram


Launch app
menu file missing/empty
Step 1: Enter name + phone
Show error, exit gracefully
No
Valid name & phone?
Yes
Step 2: Enter quantity 1-10
No
Valid integer in range?
Yes
qty >= 5 ? flag 10% discount


Step 3: Show bases, pick number
No
In range?
Yes
Step 4: Show pizzas, pick number
No
In range?
Yes
Step 5: Show toppings, pick number
No
In range?
Yes
Compute bill: subtotal -> discount -> GST 18%



5. Drawbacks Analysis (honest limitations)

This system, **exactly as specified**, has real limits:

1. **One pizza composition per order.** The spec models a single base+pizza+topping unit multiplied by quantity. A real order ("2 Margherita + 1 BBQ") needs a multi-line cart; ours applies the same composition to all N pizzas.
2. **Flat-file persistence does not scale.** `orders_log.txt` works for a demo but at **1,000 orders** it has no indexing, no concurrent-write safety (two simultaneous orders can interleave/corrupt a line), no querying, and no backup. This is the core reason Stage 3 moves to Postgres/Supabase.
3. **No inventory or availability.** The menu loads from a file; nothing tracks stock-outs, so a customer can order a sold-out item.
4. **No real payment.** "Payment" is a selected mode + confirmation string — no gateway, no settlement, no idempotency, no refunds.
5. **No authentication / abuse protection.** Anyone can place orders; no OTP on the phone number, no rate limiting, no fraud checks.
6. **Discount logic is shallow.** A blanket 10% at qty ≥ 5 with no minimum order value or per-customer cap is exploitable and may erode margin (see `BUSINESS_ECONOMICS.md`, Q4).
7. **GST is single-rate, delivery-only.** Hardcoded 18% ignores the 5% dine-in case and any ITC accounting — fine for delivery, wrong if the model expands.
8. **Single-session, no resumption.** If the browser closes mid-flow, the order is lost.

9. **No observability.** No logging/metrics/alerting beyond the orders file; failures are invisible to the owner.

What's missing for real production: a database with constraints + backups, auth + OTP, a payment gateway with webhooks, inventory, structured app logging/monitoring, automated tests in CI, and a deployment pipeline. Stages 2→3 deliberately close the biggest of these.

Future scope (beyond Stage 3). The headline next step is a **voice ordering agent**: the customer *speaks* their order and hears the assistant reply, layering browser speech-to-text and text-to-speech over the conversational ordering agent. This brings SliceMatic full circle — the outlet began with customers phoning in their orders; voice restores that natural "just talk" experience while keeping every order automated, validated, priced correctly, and logged. It reuses the same `core.py` rules (the model never does the maths) and degrades gracefully to typing where speech isn't available. Further out: multi-pizza carts, live inventory/availability, a real payment gateway, OTP authentication, and multilingual (Hindi/Hinglish) ordering. See `AI_FEATURE.md` (Option D) for the voice design.

---

6. Cost vs Value Analysis

Build effort (estimated engineering hours, team of 3–4)

Work item	Hours
Stage 1 — PRD + business model	8–10
Stage 2 — Gradio MVP (logic, validation, bill, logging, tests)	18–24
Stage 3 — Next.js on Vercel (UI + full flow)	22–30
Stage 3 — Supabase schema, auth, admin dashboard, CSV export	16–22
Stage 3 — AI feature (OpenRouter agent) + README/system prompt	10–14
Demo prep, Loom, hardening	8–10
Total	**≈ 82–110 hrs**

Measurable value to SliceMatic

- **Operational efficiency:** removes ~1–2 min of manual transcription + maths per call. At 47 orders/day that is ~60–90 staff-minutes/day reclaimed, fewer wrong orders, and faster turnaround — counter staff can be redeployed.
- **Revenue / margin:** validated pricing + correct GST eliminates billing errors; the bulk discount nudges basket size; data enables menu engineering toward high-margin combos.
- **Data asset:** every order captured (item mix, AOV, time-of-day, payment split, repeat customers) — the foundation for the forecasting/recommendation AI features and for the three BI metrics in `BUSINESS_ECONOMICS.md` Q6.
- **Customer experience:** consistent, fast, self-service ordering with a transparent bill; the AI layer adds personalised suggestions / natural-language ordering.

Verdict: at a contribution margin of ~Rs.644/order, the build pays for itself quickly if it lifts conversion or basket size even marginally and prevents order errors — the durable data asset is worth more than the hours spent.

SliceMatic — Business Unit Economics

FDE Programme · Batch 2487 · Stage 1, Part B Built on the FY 2024–25 reference model; numbers we **adopt, challenge, or correct** are flagged.

> **Costing convention (decided up front):** menu prices are **GST-exclusive**. GST is added at billing, matching the assignment spec and the reference sample bill. See *Reconciliation notes* for the two places the reference doc contradicts this.

---

1. Monthly Fixed Costs

Incurred regardless of volume. We adopt the reference figures.

Item	Monthly (Rs.)
Kitchen rent (1,200 sq ft)	55,000
Equipment EMI (Rs.4.5L, 36-mo)	14,500
Electricity (~1,800 units)	12,600
Gas / LPG (3 cylinders)	5,550
Head chef	28,000
Kitchen helper	16,500
Counter + billing staff	18,000
Delivery riders (2, fixed component)	32,000
Internet + POS + SaaS	2,800
Marketing (fixed)	8,000
Packaging (fixed minimum)	4,200
Misc / contingency (3%)	5,760
Total fixed cost	**2,02,910**

---

2. COGS per Pizza — ingredient-level

Wholesale rates (INA Market / Azadpur Mandi, Apr 2025).

Component	Margherita (Thin)	Farm House (Thick)	Cheese Burst	Menu avg
Base ingredients	38	45	72	52
Sauce + seasoning	12	14	14	13
Cheese	28	32	65	38
Pizza-specific toppings	8	22	18	17
Add-on topping (avg 1)	10	10	10	10
Packaging (box + bag)	18	18	18	18
Delivery variable	22	22	22	22
Total COGS/pizza	**136**	**163**	**219**	**170**
Selling price (base+pizza+1 topping)	397	417	447	420
Gross margin/pizza	**261 (65.7%)**	**254 (60.9%)**	**228 (51%)**	**250 (59.5%)**

> ⚠️ **Correction we apply:** delivery (Rs.22) is genuinely **per-order**, not per-pizza — a 5-pizza order is one delivery trip. Baking it into per-pizza COGS overstates the cost of multi-pizza orders. Our contribution analysis (§5) treats delivery as a single per-order line.

---

3. Gross Margin by Category

For an average order (Rs.420 ex-GST per pizza, 1 topping):

Category	Avg revenue contribution	Avg cost	Margin
Base (Rs.149–229)	~Rs.177	~Rs.52 ingredients	~71%
Pizza (Rs.299–379)	~Rs.338	sauce + cheese + specific toppings ~Rs.68	~80%
Topping add-on (Rs.39–69)	~Rs.52	~Rs.10	~81%

Toppings and the pizza layer are the highest-margin elements — a direct lever for menu engineering (push high-margin topping attach).

---

4. Revenue Model (60% capacity, launch target)

Metric	Weekday	Weekend	Monthly
Days	22	8	30
Orders/day (60% cap)	38	68	~47 avg
AOV (ex-GST)	792	940	847
Daily revenue (ex-GST)	30,096	63,920	~40,500
Monthly revenue (ex-GST)			**≈ 11,89,000**
GST collected (18%, pass-through)			≈ 2,14,000

> ⚠️ **Reconciliation:** the reference §4 treats Rs.847 as a **GST-inclusive** figure (gross Rs.11.73L − GST Rs.1.77L = net Rs.9.96L). That contradicts §5 and the sample bill, which treat prices as **ex-GST** with 18% added on top. We standardise on **ex-GST**, so monthly revenue is ≈ Rs.11.89L (847 × ~1,404 orders) and GST is collected **on top** (≈ Rs.2.14L), not carved out. Max-capacity (80 orders/day) revenue ≈ Rs.19.6L/month.

---

5. Contribution Margin & Break-Even

Per-order variable costs (ex-GST basis):

Line	Per order (Rs.)
Revenue (ex-GST)	847
− Ingredient COGS	(148)
− Packaging (variable)	(18)
− Delivery (rider incentive, per order)	(22)
− Payment gateway (1.8% of GMV)	(15)
Contribution margin	**644 (76%)**

> Note: §5's ingredient COGS of Rs.148/order is higher than the COGS table's Rs.130 ingredient-only average — consistent with orders averaging slightly more than one pizza / a richer mix. We use the §5 breakdown as the P&L basis.

Break-even

Fixed cost / month	CM / order	Break-even / month	Break-even / day
2,02,910	644	**315 orders**	**≈ 11 / day**

At 60% capacity (47/day) the outlet runs at **~4.3× break-even** — a healthy safety margin. Monthly EBITDA at 60% ≈ Rs.7.0L (operating margin ~59%).

---

6. Delivery Radius Economics

Radius	Households	Delivery time	Trips/day/rider	Verdict
0–2 km	~18,000	8–12 min	55–60	Premium SLA
2–4 km	~55,000	15–22 min	30–35	**Sweet spot**
4–6 km	~1,10,000	25–40 min	18–22	Marginal, needs 3rd rider
>6 km	—	40+ min	12–15	Not viable at launch

Recommendation: launch at **4 km** (large market, SLA intact); expand to 5 km after adding a 3rd rider (~Rs.16,000/month fixed), justified once volume crosses ~55 orders/day (see Q3).

7. GST Treatment — how 18% flows through the P&L

- Home delivery of restaurant food attracts **18% GST** (5% applies only to dine-in without ITC).
- GST is **charged to the customer** on top of the ex-GST bill and **collected on the government's behalf** — it is a **liability, not revenue**. The P&L is always computed ex-GST.
- **Input Tax Credit (ITC):** SliceMatic reclaims GST on ingredients, packaging, and equipment — est. Rs.18,000–22,000/month.
- **Net GST payable** ≈ output GST (~Rs.2.14L on the ex-GST-corrected base; ~Rs.1.78L on the reference base) − ITC (~Rs.20,000) ≈ **Rs.1.58–1.94L/month** remitted.
- **Who absorbs it?** The **customer**. GST is margin-neutral to SliceMatic; the only profit effect is the small ITC benefit. This is exactly why the ordering system shows prices ex-GST and adds GST at billing.

Reference sample bill (verified by our test suite → exact match):

Component	Amount (Rs.)
Base: Cheese Burst	229.00
Pizza: BBQ Chicken	379.00
Topping: Extra Cheese	69.00
Subtotal (1 pizza)	677.00
Qty 5 → subtotal	3,385.00
Discount 10% (qty ≥ 5)	(338.50)
Post-discount	3,046.50
GST @ 18%	548.37
Total payable	**3,594.87**

8. Challenge Questions (required analysis)

Q1 — Break-even if rent rises to Rs.70,000? At what rent is the model unviable?

Rent +Rs.15,000 → fixed = Rs.2,17,910. Break-even = 217,910 / 644 = **339 orders/month ≈ 12/day** (up from 11). Negligible at 60% capacity.

"Unviable" = break-even exceeds the sustainable plan. Setting break-even = 60%-capacity (47/day = 1,410/month) gives max fixed cost = 1,410 × 644 = Rs.9,08,040, i.e. a **rent of ≈ Rs.7.6L/month** before the launch plan stops covering costs. The model is extraordinarily rent-tolerant; rent is not the risk variable — **volume** is.

Q2 — 40% of orders via aggregator at 25% commission?

Assume aggregator orders pay 25% commission on order value and the aggregator bears delivery + payment processing:

- Aggregator-order CM = 847 − 148 (COGS) − 18 (packaging) − 211.75 (25% × 847) = **Rs.469.25**
- Direct-order CM = **Rs.644**
- Blended CM = 0.6 × 644 + 0.4 × 469.25 = **Rs.574.1 / order**
- New break-even = 202,910 / 574.1 = **354 orders/month ≈ 12/day**
- EBITDA at 47/day falls from ~Rs.7.0L to ~Rs.6.1L (**~14% lower**).

Break-even barely moves, but **profit takes a real hit** — every aggregator order is worth ~27% less in contribution. Strategic implication: aggregators buy reach but tax margin; push direct ordering (this app + the AI features) to protect contribution.

Q3 — When is a 3rd rider justified?

A 3rd rider adds ~Rs.16,000/month fixed. It pays for itself with just 16,000 / 644 ≈ **25 incremental orders/month (<1/day)**. So the constraint is **operational, not financial**: two riders saturate at ~55–66 trips/day, and with peak-hour clustering SLA degrades earlier. Hire the 3rd rider at **~55 orders/day** — by then it is both necessary for the 30-min guarantee and trivially profitable.

Q4 — Impact of the qty ≥ 5, 10% discount on a 5-pizza order. Value-driver or leak?

Using avg Rs.420/pizza (ex-GST), one delivery for the order:

	No discount	With 10% discount
Subtotal (5 × 420)	2,100	2,100
Discount	—	(210)
Post-discount	2,100	1,890
Variable cost (ingredients 650 + packaging 90 + delivery 22 + gateway ~40)	(807)	(802)
Contribution margin	**1,293**	**1,088 (57.6%)**

The discount costs ~Rs.205 of CM. But a 5-pizza order (CM Rs.1,088) is worth **~4.5× a single- pizza order** (CM ~Rs.241), partly because delivery is shared across the basket. Verdict: a **value-driver when it genuinely enlarges baskets**, a **margin leak only on customers who'd buy 5 anyway**. Recommendation: keep it, but consider 7–8% or a minimum-order-value gate, and measure incremental basket lift (Q6 metric #3).

Q5 — If COGS rises 12%, how many extra daily orders to hold EBITDA flat?

Ingredient COGS 148 → 165.76 (+17.76/order); CM 644 → **626.24**. To hold EBITDA at ~Rs.7.0L: required orders = (700,756 + 202,910) / 626.24 ≈ 1,443/month ≈ **48.1/day** — about **+1.3 orders/day (~40/month)**. The model absorbs ingredient inflation easily because variable cost is a small slice of a high-CM order.

Q6 — Three BI metrics from order data that improve profitability

1. **Hourly / day-of-week demand curve** → roster chefs and riders to demand, cutting idle labour cost and protecting the 30-min SLA (this is the Option C forecasting feature).
2. **Item attach-rate & basket/combo profitability** → menu engineering: surface high-CM base+pizza+topping combos, retire low-CM SKUs, re-price laggards.
3. **Customer RFM (recency-frequency-monetary) & repeat/churn rate** → targeted retention; stop spending discount on customers who would reorder anyway (directly addresses the Q4 leak).

---

9. Reconciliation Notes (where we depart from the reference)

1. **Ex-GST vs GST-inclusive AOV** — reference §4 carves GST out of Rs.847; §5 and the sample bill add GST on top. We standardise on **ex-GST** (spec-aligned): monthly revenue ≈ Rs.11.89L, GST collected on top.
2. **Delivery is per-order, not per-pizza** — corrected in the contribution model.
3. **Ingredient COGS** — §5 uses Rs.148/order vs the table's Rs.130 ingredient-only; we use the §5 P&L breakdown and flag the gap.

These corrections make the model internally consistent and slightly *more* favourable than the reference, while keeping every number defensible.